

AADK 7: Task 3 — Comprehensive Feature Design & Implementation Strategy

Student Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin – Session 7

USN: 1hk22cs115

email: 1hk22cs115@hkbk.edu.in

1. Feature Definition

Feature: Bill Split & Tip Tracker

The **Bill Split & Tip Tracker** allows users to enter a bill amount, number of people, and tip percentage to calculate the total tip and per-person payment. Users can also toggle a round-up option and save the result in a history list for future reference.

This feature is useful for groups dining together or sharing expenses because it quickly calculates fair contributions and keeps a record of previous bill splits.

Target Users

- Friends splitting restaurant bills
- Travelers sharing expenses
- Families managing shared payments

Key Behaviors

- User inputs bill amount and number of people
 - User selects tip percentage
 - Toggle option for rounding the tip
 - Calculate per-person amount
 - Save calculation to a history list
-

2. Implementation Strategy Document

Layout Sketch

Main Screen

Bill Split & Tip Tracker

Bill Amount

[TextField]

Number of People

[TextField]

Tip Percentage

[Slider / TextField]

Round Up Tip

[Switch]

[Calculate] [Save] [Clear]

Result

Tip Amount: ₹20

Per Person: ₹30

History Screen

History

Bill ₹100		People 4		Tip 20%		Per Person ₹30
Bill ₹250		People 5		Tip 10%		Per Person ₹55
Bill ₹500		People 3		Tip 15%		Per Person ₹191

Button List and Actions

Button	Function	State Variables Affected
Calculate	Calculates tip and per-person split result,	tipAmount
Save	Saves calculation to history list	historyList
Clear	Clears all inputs and results	billInput, numPeople, result

State Model

State Variable	Type	Mutable / Derived	Initial Value	Purpose
billInput	String	Mutable	""	Stores bill amount input
numPeople	Int	Mutable	1	Number of people splitting bill
tipPercent	Int	Mutable	15	Tip percentage
roundUp	Boolean	Mutable	false	Whether tip should round up
result	Double	Derived	0.0	Final calculated amount
historyList	List	Mutable	emptyList()	Stores saved split history

Example State Initialization

```
var billInput by remember { mutableStateOf("") }
var numPeople by remember { mutableStateOf(1) }
var tipPercent by remember { mutableStateOf(15) }
var roundUp by remember { mutableStateOf(false) }

val result by derivedStateOf {
    calculateTip(billInput, tipPercent, numPeople, roundUp)
}
```

Pseudo-Code / Key Logic

Save Action (Update History List)

```
data class SplitHistory(  
    val bill: Double,  
    val people: Int,  
    val tipPercent: Int,  
    val perPerson: Double  
)  
  
var historyList by remember { mutableStateOf(listOf<SplitHistory>()) }  
  
fun saveSplit() {  
    val newEntry = SplitHistory(  
        bill = billInput.toDouble(),  
        people = numPeople,  
        tipPercent = tipPercent,  
        perPerson = result  
    )  
  
    historyList = historyList + newEntry  
}
```

3. Testing & Edge Case Plan

Test Case 1 — Valid Input

Input	Expected Output
bill = 100	-----/-----

Input	Expected Output
numPeople = 4	-----/-----
tip = 20%	-----/-----
roundUp = false	per person = ₹30

Explanation:

Tip = 20 → Total = 120 → Per person = 30.

Test Case 2 — Invalid Input

Input	Expected Behavior
bill = -10	Error message shown
bill blank	Calculate button disabled

UI Feedback:

- Error text displayed
 - Red border around TextField
-

Test Case 3 — Edge Case

Input	Expected Behavior
bill = 10000	calculation works normally
numPeople = 0	Show validation message

Handling Strategy:

- Force minimum value of numPeople = 1
 - Prevent division by zero.
-

UI Test Considerations

- **Orientation Change:** state should remain after screen rotation.
 - **Keyboard Handling:** keyboard should hide when calculate button is pressed.
 - **State Persistence:** use `rememberSaveable()` for inputs to survive configuration changes.
-

4. Scaling & Maintenance Considerations

To scale this feature, support for **multiple currencies** could be added using a currency selector and formatting utilities. The history list could also be stored in **DataStore or Room database** to persist data between app sessions. Calculation logic should be separated into reusable functions to keep the composables clean and maintainable. State management could also be moved to a **ViewModel** for better architecture.

5. Reflection

The most complex part of planning this feature was organizing the different state variables and ensuring they interact correctly without causing incorrect UI updates. This exercise improved my understanding of how state flows through composables and how derived state simplifies calculations. I also learned the importance of validating inputs before performing operations like division. In future implementations, I will avoid placing too much logic inside composables and instead separate business logic into functions or ViewModels.